



Универзитет „Св. Кирил и Методиј“ во Скопје
Факултет за информатички науки и компјутерско инженерство

ДИПЛОМСКА РАБОТА

Автоматизација на процес за менадирање неструктурирани податоци со користење на големи јазични модели

Ментор

Проф. д-р Ана Мадевска Богданова

Кандидат

Дејан Ристовски
211175

Скопје, март 2026

Апстракт

Процесирањето на неструктуриран текст во структуриран формат претставува значајна област во рамки на машинското учење и развојот на големите јазични модели. Нивната способност да анализираат и обработуваат огромни количини податоци за кратко време претставува една од нивните најголеми предности, но воедно и предизвик кој отвора простор за истражувања и унапредувања. Со големите достигнувања во областа на големите јазични модели во последните години, се отвори можност за значително подобрување на автоматското процесирање на текстуални податоци, создавање на паметни системи за препознавање и мапирање на информации, како и за креирање на алатки кои можат да поддржат донесување поефикасни и попрецизни одлуки во различни домени.

Целта на овој труд е да понуди решение за автоматизација на процесот за менаџирање на вештини и таленти на вработени со помош на големите јазични модели. Секоја вештина може да има повеќе нивоа, со која што се добива хиерархија на нивото на познавање. Вредноста на автоматизацијата на ваков систем уште повеќе е значајна бидејќи овозможува брзо, прецизно и објективно следење и ажурирање на вештини, намалувајќи ја потребата за мануелна работа и подобрувајќи ја ефикасноста при управување со талентите на вработените.

Апликацијата користи еден унифициран процесен тек (pipeline) за да постигне три клучни типови на автоматизација. Прво, системот овозможува генерирање на нови вештини кои може да се внесат во базата на податоци, со што се овозможува постојано ажурирање и проширување на знаењето на организацијата. Второ, истиот процесен тек се користи за процесирање на CV на вработени, овозможувајќи детекција на вештини кои се подобруваат, ослабнуваат или недостасуваат, со што се добива јасна слика за развојот на кадрите. Трето, системот автоматски ги препознава потенцијалните нови вработени кои аплицираат на огласи, споредувајќи ги нивните вештини со барањата на позицијата, овозможувајќи побрзо и пообјективно селектирање на кандидати. На овој начин, еден процесен тек ефективно ги интегрира и автоматизира сите овие процеси, зголемувајќи ја ефикасноста и конзистентноста во менаџирањето на талентите.

Содржина

Апстракт	2
1. Вовед.....	5
2. Методологија.....	7
2.1. Користени технологии	7
2.1.1. Backend.....	7
2.1.2 Frontend	10
2.1.3. Langchain4j.....	11
2.2. Структура на податоци	12
2.2.1. EP дијаграм	12
2.2.2. Семантичка структура на вештини и нивоа	13
2.3. Имплементација на процесен тек за процесирање на текст	13
2.3.1. Користење LLM агенти vs. embedding на текст	13
2.3.2 RAG системи	14
2.3.3. Имплементација на процесен тек	15
2.3.4. Потреба за повеќе агенти	17
2.3.5. Примена на процесниот тек во различни сценарија.....	18
2.3.6. Генерирање на огласи.....	18
3. Апликација	19
3.1. Преглед на апликацијата.....	19
3.2. Кориснички улоги и автентикација	20
3.2.1. Нерегистрирани корисници	20
3.2.2. Вработени	21
3.2.3. Менаџери	21
3.3. Функционалности	21

3.3.1. Регистрација и најавување на корисник.....	21
3.3.2. Регистрација на корисник.....	22
3.3.3. Најава на корисник.....	22
3.3.4. Преглед на корисници во системот	23
3.3.5. Преглед на кориснички профил	23
3.3.6. Менаџирање на вештини.....	24
3.3.7. Процесирање на резиме и менаџирање на унапредени вештини.....	24
3.3.8. Преглед на вештини (Skills dashboard)	25
3.3.9. Процесирање на текст и менаџирање на вештини.....	25
3.3.10. Генерирање на оглас за работа	26
3.3.11. Преглед на огласи за работа	27
3.3.12. Преглед и менаџирање на оглас за работа.....	27
3.3.13. Аплицирање на оглас за работа.....	28
3.3.14. Преглед на корисници кои аплицирале на оглас	28
4. Заклучок	29
5. Библиографија.....	30

1. Вовед

Од појавата на интернетот до денес, количината на податоци околу нас значително се зголеми. Опкружени сме со различни електронски уреди кои постојано собираат и складираат информации, создавајќи дигитални профили за нашите навики и карактеристики. Секој пат кога посетуваме веб-страница, нашите активности се следат и анализираат, а собраните податоци често се користат и на други платформи за прикажување персонализирани препораки и реклами.

Во денешно време, сè почесто се користи вештачката интелигенција за пребарување информации. Многу задачи за кои пред неколку години ни беа потребни значително време и труд, денес можеме да ги завршиме со внесување на неколку клучни зборови. Доколку порано „примитивно“ пребарувавме научни трудови преку отворање на повеќе страници и детално читање на текстови, денес доволно е да ги формулираме нашите барања и веднаш да добиеме најрелевантни резултати. Вештачката интелигенција претставува нова технолошка револуција што веќе има значително влијание врз секојдневниот живот и работата во различни области.

Од овие примери можеме да ја согледаме една од најголемите придобивки на вештачката интелигенција во нашиот секојдневен живот — способноста да ги трансформира неструктурираните податоци во структуриран и лесно разбирлив формат. Наместо да трошиме време прелистувајќи стотици документи во потрага по конкретна информација, денес можеме да се потпремe на модели кои тоа го прават за само неколку секунди. Реалноста е дека големите јазични модели имаат значителна предност благодарение на нивната способност да обработуваат огромни количини податоци и да извлекуваат релевантни сознанија со голема брзина и ефикасност.

Процесирањето на големи корпуси на текст претставува предизвик кој се адресира уште одамна во рамките на машинското учење. Со текот на времето се развија модели способни за споредување на текстови, препознавање на именувани ентитети и класификација на содржини. Сепак, и покрај овие достигнувања, прецизното толкување на корисничките барања остана ограничено сè до појавата на големите јазични модели.

Почетоците на овие модели се поврзуваат со познатиот труд за трансформер архитектурата [1], кој вовеле нов пристап во обработката на текст. Клучна компонента на оваа архитектура е attention механизмот, кој овозможува моделите да се фокусираат на релевантните делови од влезните податоци. Токму овој механизам претставува основа

на современите модели што денес се широко користени во различни апликации и секојдневни задачи.

Целта на овој труд е да се искористат овие способности за обработка на големи корпуси на текст преку дефинирање на конкретни правила што моделот треба да ги следи. Со примена на големи јазични модели, како и модели за споредување на текстови, овозможено е процесирање на податоци од неструктуриран формат и нивно трансформирање во јасно дефинирана структура.

Трудот се фокусира на примена на овој пристап во доменот на менаџирање на таленти и вештини кај поединци поврзани со одредена фирма, каде што трансформацијата на податоци од неструктуриран во структуриран формат може значително да ја унапреди организацијата, анализата и искористувањето на информациите.

Мотивацијата за избор на овој домен произлегува од лично искуство во развој на систем за менаџирање на вештини и тренинзи за вработени. Во рамки на една организација, секој вработен поседува специфичен сет на вештини кои се користат за задоволување на барањата на различни работни позиции и за оптимална распределба на задачите. Овие вештини најчесто се сумираат во кратка форма – резиме или CV.

Сепак, резимето претставува неструктуриран тип на податок, односно текстуална содржина без јасно дефинирана структура, што го отежнува неговото автоматско процесирање и анализа. Историјата на обработката на ваквите податоци може да се подели во неколку клучни фази:

1. Пред појавата на компјутерите, овој проблем се решавал рачно – преку читање на сите резимеа на вработените, меморирање на нивните вештини и креирање стратегија за нивна распределба. Иако овие информации можеле да се запишат на хартија во одреден структуриран формат за полесно управување, ваквиот пристап бил ограничен и неефикасен. Со зголемување на бројот на вработени, особено кога станува збор за стотици луѓе, ваквото рачно менаџирање станува практично невозможно.
2. Со појавата на компјутерите се развиваат системи способни за складирање и управување со големи количини податоци. Ова овозможува креирање на бази во кои можат да се внесат вештините на илјадници вработени од различни сектори. Пребарувањето и филтрирањето според конкретни вештини станува значително поефикасно, што доведува до подобро и поорганизирано управување со човечките

ресурси. Како резултат на тоа, ефикасното менаџирање на илјадници работни позиции станува остварливо.

3. Со појавата на вештачката интелигенција се развиваат системи кои можат да обработат голем број документи за само неколку секунди. Пребарувањето на вештини се поедноставува преку користење на природен јазик, наместо рачно селектирање на специјализирани филтри. Теоретски, внесувањето на вештините од вработените во базата може да се автоматизира, што го елиминира потребното време за рачно читање и обработка на резимеата.

Во овој труд се предлага решение за управување со вештините на вработените во една компанија преку автоматизација на процесот на трансформација на резимеата во структуриран формат со дефинирани вештини и нивни нивоа. Воведувањето на вакво решение значително го скратува времето потребно за менаџирање на сегашните вработени, како и на потенцијалните кандидати кои аплицираат за одредени работни позиции.

2. Методологија

2.1. Користени технологии

2.1.1. Backend

Во оваа апликација backend слојот се грижи за комуникација со базите на податоци како и извршување на повиците потребни за добивање одговор од моделите користени за процесирање на податоците. Целиот backend слој е граден со Spring Boot, додека како библиотека за полесно градење на AI агенти се користи Langchain4j. Бидејќи се користи комбинација од релациони податоци и текст со embedding вектори, се користат две бази на податоци PostgreSQL и Qdrant.

Spring Boot

Spring Boot [5] е еден од најпознатите frameworks за градење на апликација која подржува извршување на API повици. Spring boot доаѓа со многу познати библиотеки за развој на веб апликации од кои во апликацијата се користат:

- Spring Data JPA – олеснува извршување на CRUD операции поврзани со релациони податоци и се грижи за мапирање на java објекти во релации
- Spring Webflux – се користи за градење на реактивна комуникација и имплементација на Server-Sent Events (SSE) за стриминг на податоци во

реално време од серверот до клиентот

- Spring Security – корисен за имплементација на авторизација и автентикација
- Java JWT – за полесно креирање и читање на JWT токени
- PostgreSQL – за креирање на комуникацијата меѓу Spring Boot и PostgreSQL базата на податоци
- Mapstruct – користен за едноставно мапирање помеѓу различни објекти во апликацијата (пр. DTOs, и entity објекти)
- Langchain4j – библиотека користена за градење на AI агенти. Дава алатки за prompt engineering, tool calling и парсирање на податоци
- Langchain4j OpenAI – екстензија на Langchain4j која овозможува интеграција на модели од OpenAI
- Qdrant Java client – креира комуникација помеѓу Spring Boot и Qdrant база за податоци со embedding вектори

Предноста на користење на Spring Boot е тоа што овозможува лесно инјектирање на зависностите во различни компоненти на апликацијата. Преку креирање на beans, Spring Boot знае да ги креира потребните објекти и да ги инјектира во зависност од тоа каде е потребно.

PostgreSQL

PostgreSQL [2] е open-source база на податоци која се грижи за менаџирање на релациони бази на податоци. Има поддршка за извршување на комплексни квериња, трансакции и типови на податоци.

Со користење на Spring Data JPA библиотеката, една табела се дефинира во обичен Java објект и се дефинираат различните компоненти со помош на анотации. Понатаму обврската за креирање на DDL кверињата за менаџирање на структурата на базата е оставена на JPA и нејзината имплементација од Hibernate.

```
@Entity
public class Skill {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String description;

    @OneToMany(mappedBy = "skill", orphanRemoval = true, fetch = FetchType.EAGER)
    private List<SkillLevel> levels;
}
```

Код 1. Изглед на еден ентитет дефиниран во Spring boot

Освен за генерирањето на DDL квериња, Spring Data JPA нуди можност и за лесно дефинирање на repository слој кој се грижи за пристапот до базата на податоци. Со дефинирање на интерфејси кои го прошируваат JpaRepository интерфејсот, можеме да креираме обични Java функции кои автоматски се имплементираат од страна на Hibernate.

```
public interface SkillLevelRepository extends JpaRepository<SkillLevel, Long> {  
    Optional<SkillLevel> findByNameAndSkillId(String name, Long skillId);  
}
```

Код 2. Изглед на repository имплементиран со помош на Spring Data JPA

Qdrant

Како векторска база на податоци се користи Qdrant. За разлика од релационите бази на податоци, векторските бази на податоци се користат за да се чуваат векторски репрезентации на текст. Најчесто се користат за градење на RAG системи каде е потребна семантичка споредба на податоци.

За пристапот до Qdrant базата на податоци се користи Qdrant Java Client библиотеката. За разлика од Spring Data JPA, оваа библиотека се користи само за пристап до Qdrant и сите квериња кои што се извршуваат треба да ги напишеме преку код. Секој податок во Qdrant се чува како точка (point) која што се состои од векторот и дополнителни metadata податоци.

```
Embedding queryEmbedding = embeddingModel.embed(query).content();  
  
List<Points.ScoredPoint> searchResults = qdrantClient.searchAsync(  
    Points.SearchPoints.newBuilder()  
        .setCollectionName(collection.getValue())  
        .addAllVector(queryEmbedding.vectorAsList())  
        .setLimit(limit)  
        .setWithPayload(Points.WithPayloadSelector.newBuilder()  
            .setEnable(true)  
            .build())  
        .build()  
).get();
```

Код 3. Изглед на едено квери напишано преку код со помош на Java Qdrant Client

Првата векторска репрезентација на текст се појавува со развојот на моделот Word2Vec [3]. Со користење на векторска репрезентација, текстот се претвора во високо-димензионален простор каде што сличните текстови се наоѓаат блиску еден до друг. Ова може да се визуелизира преку намалување на димензиите на векторите и нивно прикажување во координатен систем. Една од популарните техники за ова е t-SNE [4], која овозможува визуелизација на податоци со одржување на релативната сличност помеѓу текстовите.

2.1.2 Frontend

Frontend-от на апликацијата е развиен со помош на Angular [6], кој е open-source framework развиен од Google. Angular нуди комплетен сет на алатки и функционалности, како што се компоненти, routing, dependency injection и управување со форми. Дополнително, во проектот се користи Angular Material, библиотека која обезбедува готови UI компоненти базирани на Material Design. Таа овозможува брзо креирање на модерен и конзистентен кориснички интерфејс преку компоненти како што се копчиња, табели, дијалози и форми.

Апликацијата е изградена врз принципот на компонентна архитектура, каде што секој дел од интерфејсот и логиката е организиран во независни, повторно користливи единици наречени компоненти. Главните компоненти од кои е изградена апликацијата се следниве:

- App – главната компонента која што е на врвот на дрвото од компоненти која е одговорна за генерирање на различните компоненти во зависност од рутата
- Navbar – ја обезбедува основната навигација низ апликацијата
- LoginForm и RegisterForm – компоненти кои што се одговорни за најавување и регистрирање на корисници
- Profile – компонента која прикажува информации за одреден корисник како и функционалност на дополнување на профил според процесирани резиме
- SkillDashboard – компонента која што ги прикажува сите вештини кои што се наоѓаат во системот
- SkillExtract – ја држи функционалноста да процесира текст и генерира нови вештини кои може да се додадат во системот
- UsersDashboard – компонента која што ги претставува сите нови корисници како и тие што се потенцијални за вработување
- JobPostingsDashboard – ги држи огласите за работа на кои што можат да аплицираат кандидати

Превземање на генерирани вештини со Server-Sent Events (SSE) и WebFlux

Покрај обичните HTTP повици кои се правени од frontend-от кон backend-от, имплементирано е и превземање на генерираните вештини со помош на Server-Sent Events. На овој начин корисниците ги добиваат податоците преку асинхронизирани барања што овозможува прикажување на резултатите во реално време, без потреба од чекање на целосниот одговор од серверот.

Дополнително, користењето на SSE придонесува за подобрување на корисничкото искуство, бидејќи овозможува континуирано ажурирање на податоците без потреба од повторно иницирање на барања од страна на клиентот. Ова го намалува оптоварувањето на серверот и мрежата, а воедно обезбедува поефикасна и побрза комуникација, особено при обработка на податоци кои се генерираат постепено.

При иницирање на барањето од страна на frontend апликацијата, се воспоставува SSE конекција кон соодветен endpoint на backend-от. Барањето се обработува со помош на Spring WebFlux, кој овозможува реактивен и неблокирачки пристап при извршување. Наместо да чека комплетна обработка, backend-от враќа stream од податоци (Flux), каде што секоја генерирана вештина се испраќа како посебен настан.

На клиентската страна, воспоставената SSE конекција се користи за континуирано примање на податоци од серверот. Апликацијата го обработува пристигнатиот stream на настани и ги трансформира во соодветни објекти, кои потоа се прикажуваат во корисничкиот интерфејс. Овој пристап овозможува постепено прикажување на резултатите и динамичко ажурирање на податоците, без потреба од дополнителни HTTP барања или освежување на страната.

2.1.3. Langchain4j

LangChain4j претставува Java имплементација на популарната LangChain библиотека, која првично е развиена во Python. Таа овозможува лесна интеграција на големи јазични модели (LLMs) во Java апликации, преку обезбедување на апстракции за работа со промптови, обработка на текст и поврзување со надворешни AI сервиси.

Изборот на оваа библиотека е мотивиран од сè поголемата потреба за развој на апликации со вештачка интелигенција во Java екосистемот, како и потребата за конзистентност со backend технологиите користени во проектот. LangChain4j овозможува поедноставен развој на функционалности како што се генерирање на текст, обработка на природен јазик и интеграција со модели како OpenAI, при што се задржува чиста и

модуларна архитектура.

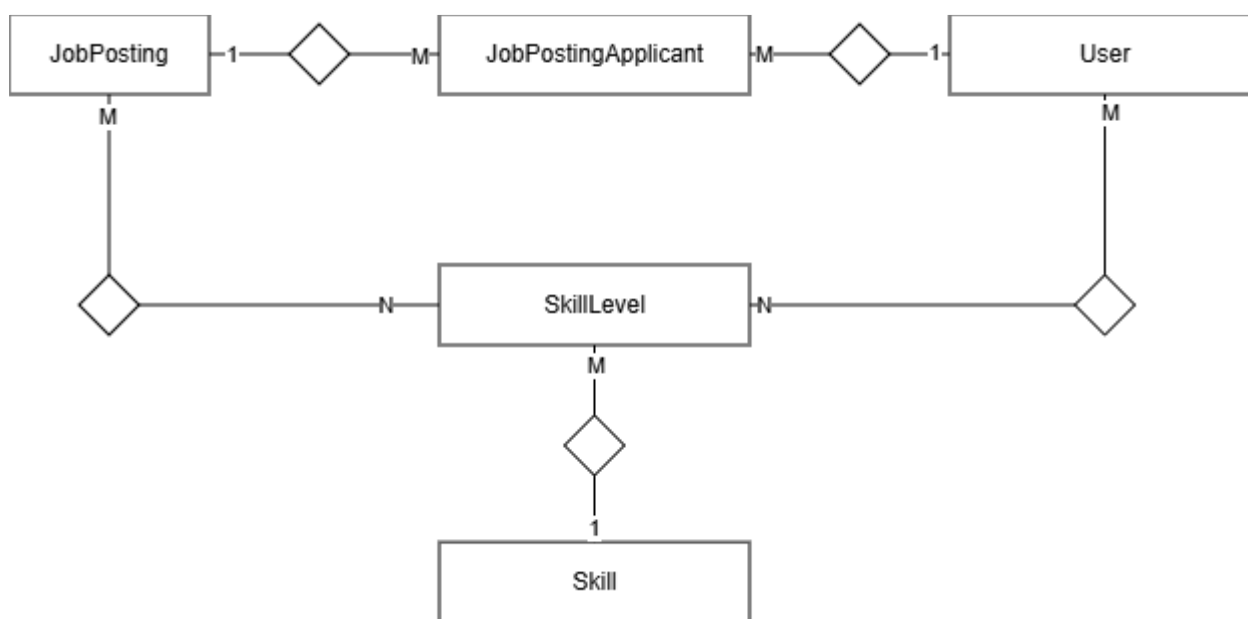
Иако LangChain4j најчесто се користи за креирање на агенти кои повикуваат различни алатки (tools), во овој проект таа се користи главно за добивање на структуриран одговор од моделите. Библиотеката овозможува да дефинираме Java објекти кои ја претставуваат структурата во која агентот треба да одговори. Оваа апстракција ни помага да се фокусираме на главната архитектура на софтверското решение и да ја оставиме логиката за комуникација со моделите и обработката на податоците на библиотеката, со што развојот станува побрз и поефикасен.

2.2. Структура на податоци

2.2.1. ЕР дијаграм

Системот се состои од неколку модели во базата на податоци од кои:

- Skill – модел кој ги чува главните карактеристики на една вештина како име и опис.
- SkillLevel – ниво кое што може да го поседува некој вработен во системот, одредено според описот и редоследот на нивото.
- User – корисник на системот. Може да биде вработен, менаџер или корисник кој што уште не е регистриран.
- JobPosting – претставува огласот на кој што корисниците можат да аплицираат.
- JobPostingApplicant – релација помеѓу корисник и оглас за работа, дополнително ја чува и оцената на корисникот кој аплицирал на огласот



Слика 1. ЕР дијаграм за релациите меѓу ентитетите во системот

2.2.2. Семантичка структура на вештини и нивоа

Од поголемо значење има структурата на вештини и нивоа, како и нивните описи. Секоја вештина има повеќе нивоа, и секое ниво си има соодветен опис за што значи еден вработен да поседува одредено ниво. Овие описи треба да се добро специфицирани за да се добие структура каде секое ниво точно одредува што е потребно за да се достигне. Пример за вештина во системот е:

- Вештина: Spring Boot
 - Ниво 1: Beginner – Can build simple Spring Boot applications using Spring Web for REST APIs and Spring Data JPA with in-memory databases. Familiar with core annotations like `@RestController` and `@Autowired`
 - Ниво 2: Intermediate – Develops structured applications using Spring Web, Spring Data JPA, Spring Validation, and Spring Boot Actuator. Handles exception management, configuration properties, and layered architectures
 - Ниво 3: Advanced – Builds complex, production-ready applications using Spring Security, Spring Cache, Spring Batch, Spring Cloud, and Actuator. Can optimize performance, implement asynchronous processing, and leverage advanced Spring Boot features for scalable systems

Од описите може веднаш да се дефинира дали одреден вработен поседува одредено ниво на вештината Spring Boot. Во овој случај тоа е опишано според познавањето на различни библиотеки кои што се користат во Spring Boot. Во други случаи е добро да се опишат и бројот на сертификации кои што треба да ги поседува, дури може да се спомнат и специфични сертификации.

Овој тип на добро структурирана хиерархија на вештини му помага на моделот да одреди на кое ниво припаѓа еден вработен. Доколку различните нивоа на вештините се дефинирани нејасно и со метрики кои што не можат да се измерат, системот може да направи погрешна проценка, да ги меша различните нивоа, или да дава непоследователни резултати. Добро структурирана хиерархија со специфични, измерливи вештини и поврзани технологии или алатки овозможува моделот да го процесира текстот и да одреди со поголема прецизност на кое ниво припаѓа еден вработен.

2.3. Имплементација на процесен тек за процесирање на текст

2.3.1. Користење LLM агенти vs. embedding на текст

Едно од најпознатите начини за споредување на текст е да се искористи споредбата на embedding на текстови. Со тоа се добиваат текстови кои што се слични еден на друг и се состојат од слични зборови. Проблемот со овој пристап е тоа што не го споредуваме

семантичкото значење еден со друг, два текста може да имаат слични зборови но да даваат сосема различно значење.

Затоа користењето на LLM е од големо значење. Способноста на овие модели е да го разберат семантичкото барање на корисниците и да изгенерираат содржина во однос на потребите. Истиот случај може да се види и ако го искористиме да мапира одредено резиме на некоја личност, со примерот за вештина на Spring boot. Моделот може да го прочита резимето и да одреди дали ги поседува одредените библиотеки кои што се опишани во одредено ниво.

Дополнително, моделот може да препознае суптилни нијанси и контекстуални разлики, како тоа дали кандидатот навистина ја користел технологијата во практична апликација или само ја споменал. Ова овозможува многу по-точно мапирање на вештини и нивоа на знаење, отколку со едноставно споредување на зборови. Исто така, LLM може да се користи за создавање хиерархии на вештини, автоматско категоризирање и препораки за надградување на знаењето, со што се зголемува прецизноста и корисноста на системите за обработка на човечки ресурси.

2.3.2 RAG системи

LLM моделите се тренирани на масивен корпус на текстови, но нивната слабост е дека не ги имаат податоците и информациите кои се случиле после тренирањето. Ако моделот е трениран со податоци до 2024-та година, тогаш нема да има познавање за ништо во 2025-та и понатаму. Исто така, доколку направиме наши податоци, како тие со вештини и нивоа, моделот нема да знае за нив доколку ние не му ги дадеме во контекст.

Со тоа помагаат RAG системите кои се направени да се даде доволно контекст на моделот за да ја изврши потребната задача. Тие се системи кои што се базираат на embedding на текст и барање на слични текстови од корпусот во базата на податоци. Во овој систем корпусот на embedding текстови се состои од податоците за вештини и нивоа, како и нивните описи. Со ова овозможуваме функционалност каде што на моделот можеме да му дадеме слични вештини и според тоа да донесе одлука за тоа како да ги изгенерира и мапира податоците.

Овие системи се користат за снабдување на агентите во процесот на обработка на текст со релевантен контекст, овозможувајќи им на моделите да донесуваат точни одлуки без да мораат самостојно да ги обработуваат сите податоци. На тој начин, агентите можат побрзо да ги идентификуваат важните информации и да ги интегрираат во структурата на вештини и нивоа, што ја зголемува ефикасноста и прецизноста на целокупниот процес.

2.3.3. Имплементација на процесен тек

Целиот процес на генерирање на вештини и нивоа може да се подели на неколку чекори и тоа:

1. **Корисникот прикачува текст/документ** – корисникот треба да прикачи текст или документ во форма на pdf
2. **Поделба на документ/текст** – моделот многу подобро го процесира документот кога е поделен во помали сегменти, истиот затоа се дели на секои 1000 карактери
3. **Генерирање потенцијални имиња за вештини** – процесот продолжува со генерирање на имиња на потенцијални вештини кои треба да се извадат од сегментот. Во prompt-от на моделот е специфицирано да не генерираат имиња со нивоа и да се генерираат само вештини во нивната најосновна форма. Како влез е даден само текст сегментот од кој што треба да генерира.

```
You are a skill extractor. Given the data extract names for the skill that are
contained in the chunked text.
These skills will be used in an ERP system for skill and competency organization.
The skills need to be something an employee can have.

Extract all skills mentioned, including:
- Technical skills (programming languages, tools, frameworks, software)
- Soft skills (communication, leadership, teamwork)
- Domain knowledge (finance, marketing, healthcare)
- Certifications or methodologies (Agile, PMP, AWS)

Generating rules:
1) Never generate skill names with levels in the name.
2) Never generate skills that you are not sure exist in the given text.
3) If no useful skills can be extracted return empty list.
4) The skill names need to be generic and not specific to the given text.
5) Try to generate the name in its most base form
6) Generate maximum 5 skills
```

Код 4. System message на агентот за генерирање имиња на вештини

4. **Генерирање вештини со нивоа и опис** – во овој чекор се генерираат нивоа за секоја вештина што е создадена во претходниот чекор. Како влез на моделот се користи името на вештината и контекстот (текстуалниот сегмент). Моделот ги генерира сите нивоа за вештината, дури и ако контекстот се однесува само на едно од нив. Дополнително, моделот треба да го определи на кое од генерираните нивоа припаѓа дадениот текст сегмент.

You are a generator for levels of a skill. You are given a skill name and context. The context is the text that you need to use to extract the levels for that skill.

Follow these rules when creating levels:

1) Every level needs the following data:

- * name - simple and precise (ex. Beginner, Intermediate, Advanced)

- * expectation - summary of max 255 characters what it is expected from employees that have this level of the skill

- * levelOrder - the number of level that this skill is (ex. 1, 2, 3)

2) All levels for a skill need to have a clear path for progression defined in the level expectations

3) The skill description needs to be generic description of what an employee needs to posses to have this skill

- * Don't include the context in the skill description, try to make it generic

4) YOU MUST generate all levels for skill:

- * 'name': the name of the new skill

- * 'levels': generate all the levels for that skill even if the context is only for a certain level

- * 'basedLevel': the name of the level that corresponds to the given context

This entities will be used in an ERP system for skill and competency organization. Always follow the schema given to you when generating an output.

Код 5. System message на агентот за генерирање на вештини со нивоа и описи

- 5. Пребарување за слични вештини во системот** – од генерираните вештини и нивните описи се проверува дали веќе постои слична вештина во базата. Споредбата се врши според сите нивоа за да се избегне појавата на различни имиња за исти барања. Доколку се најде вештина со висока сличност, се задржува само онаа со најголема оценка, и тоа само ако сличноста надминува 0.9.
- 6. Извршување на агент за поврзување на нивоа** – доколку се најде слична вештина, се повикува нов агент чија задача е да се обиде да го мапира текстуалниот сегмент на соодветно ниво од постоечката вештина. Ако текстот одговара на некое ниво, резултатот се враќа кај корисникот; во спротивно, системот продолжува со следниот чекор. Агентот добива како влез: името на вештината, контекстот (текстуалниот сегмент) и целиот опис на вештината со сите нејзини нивоа.

You are a matcher for given skill to existing skills in the context. You are given a skill name and context.

You need to either match the given skill to some level that exists in the system, or return -1 if it can't be matched. Use the context to decide the matched level.

If the level matches the skill but not the levels, return -1. If no level is matched return -1. Never invent new level ids and never return skill ids.

Don't match the level if you are not sure, just return -1.

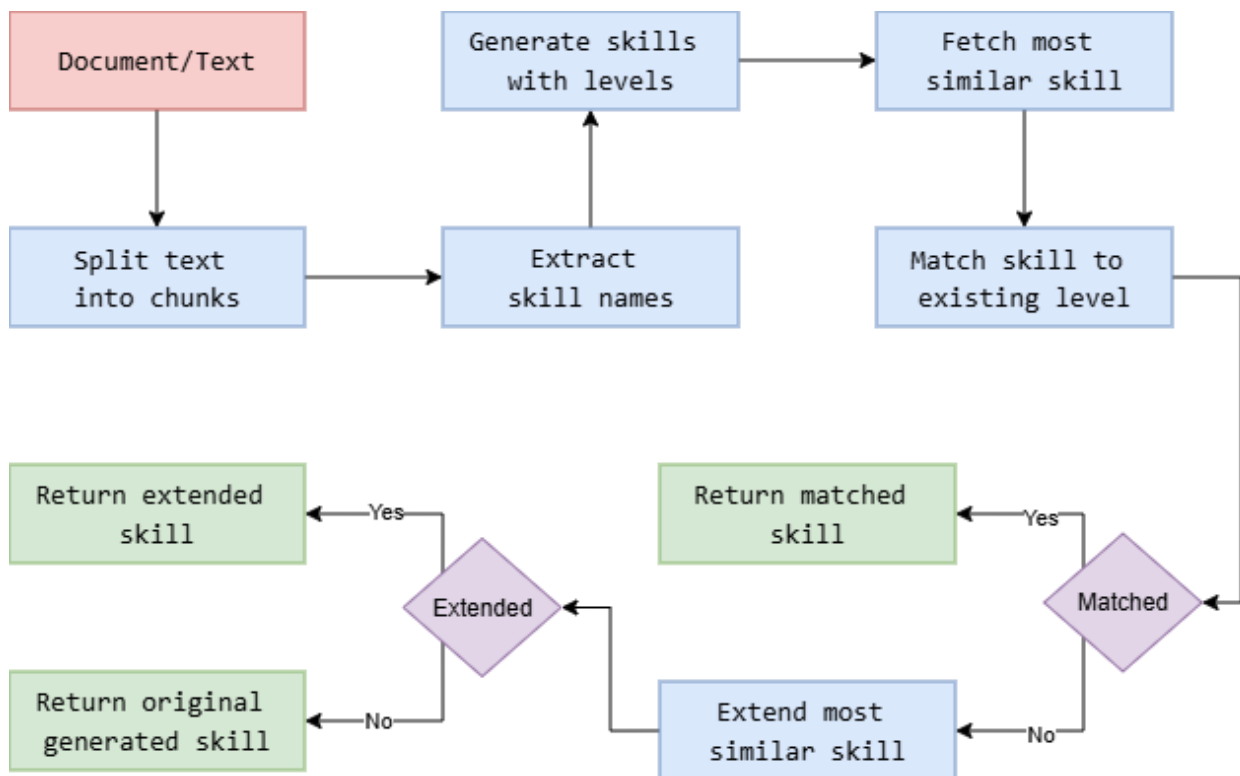
Only return the level id.

Код 6. System message на агентот за мапирање на вештини

7. **Извршување на агент за проширување на вештина со ново ниво** – овој агент има задача да ја прошири постоечката вештина со ново ниво, доколку тоа е возможно. Доколку се утврди дека вештината може да се прошири, системот ја враќа обновената вештина; во спротивно се враќа оригиналната вештина која била генерирана пред почетокот на споредбата со постоечките вештини. Како влез, агентот добива: името на вештината, контекстот (текст сегментот) и целиот опис на вештината со сите нивоа.

```
You are a skill extender. Given the skill name and context you need to decide whether the existing skill can be extended with a new level. Example you can add a level between existing levels, a new level that is at the bottom or new level that is the highest. If the skill can be extended generate the new level. Don't extend the skill if you are not sure, just return the object with null values. If the skill cannot be extended then return the object with null values.
```

Код 7. System message на агентот за проширување на вештини



Слика 2. Flowchart на процесен тек за процесирање на текст

2.3.4. Потреба за повеќе агенти

Во рамки на предложениот процесен тек, поделбата на процесот на повеќе специјализирани агенти претставува клучна архитектонска одлука која значително ги подобрува перформансите, точноста и скалабилноста на системот. Наместо еден монолитен модел кој би ги извршувал сите задачи (од екстракција на вештини до нивно мапирање и проширување), користењето на повеќе агенти овозможува јасна поделба на

одговорности и оптимизација на секој чекор поединечно.

Агентите даваат значително подобрување кога им се доделуваат специфични задачи, бидејќи секој од нив е фокусиран на решавање на еден јасно дефиниран проблем. Ова овозможува попрецизно прилагодување на влезните инструкции и контекстот, што резултира со поквалитетни и поконзистентни излези во секоја фаза од процесот.

Дополнително, ваквиот пристап овозможува поголема флексибилност и одржливост на системот. Секој агент може независно да се подобрува или заменува без да се наруши остатокот од процесниот тек, што го прави решението поедноставно за надградба, тестирање и проширување во иднина.

2.3.5. Примена на процесниот тек во различни сценарија

Истиот процесен тек опишан во секција 2.3.3 се применува во повеќе различни сценарија, со цел да се демонстрира неговата флексибилност и практична употребливост:

- **Процесирање текст** – системот овозможува анализа на произволен текст и автоматско генерирање на релевантни вештини заедно со соодветни нивоа на познавање.
- **Процесирање на CV** – при прикачување на CV за одреден вработен, документот се обработува и добиените вештини се споредуваат со веќе постоечките во системот. Како резултат, се идентификуваат промени во профилот на корисникот, односно вештини кои се унапредени (upskill), ослабнати (downskill) или нови.
- **Процесирање на апликанти и добивање оцена** – при аплицирање на оглас за работа, кандидатите прикачуваат CV кое автоматски се анализира. Добиените вештини се споредуваат со барањата на огласот, при што се пресметува процентуална усогласеност. Овој процент може да надмине 100% во случаи кога кандидатот поседува повисоко ниво на знаење од она што е предвидено за конкретната позиција.

2.3.6. Генерирање огласи

Освен главниот процесен тек за генерирање процесирање на текст, системот поддржува и агент за генерирање на оглас за работа. Агентот за генерирање на оглас за работа функционира така што како влез прима листа од селектирани вештини заедно со селектирано ниво на познавање за секое од нив. Врз основа на овие структурирани

податоци, моделот генерира конзистентен и професионален текст за оглас кој ги опфаќа клучните барања, одговорности и очекувани квалификации за позицијата. На овој начин се овозможува брзо и автоматизирано креирање на огласи кои се усогласени со

```
Generate a job posting based on the given skill level data.  
You are forbidden from adding any more skills or requirements other than those  
given to you.  
The job posting description is the actual job posting, use rich text to generate  
it.
```

конкретните потреби на организацијата.

Код 8. System message на агентот генерирање на оглас

3. Апликација

3.1. Преглед на апликацијата

Во оваа секција е претставена апликацијата развиена како дел од овој труд, која претставува практична имплементација на претходно опишаните концепти и методологии. Апликацијата овозможува автоматизирано процесирање на неструктурирани текстуални податоци, со фокус на екстракција, организација и мапирање на вештини во структуриран формат погоден за понатамошна анализа и употреба.

Решението е дизајнирано како систем кој интегрира повеќе компоненти и модели, со цел да обезбеди ефикасно и прецизно трансформирање на податоците. Посебен акцент е ставен на корисничкото искуство и едноставноста на користење, овозможувајќи прикачување на документи и добивање на структурирани резултати без потреба од мануелна интервенција. Преку оваа апликација се демонстрира како теоретските концепти можат да се применат во реално сценарио.

Бидејќи целта на трудот е да се автоматизира процес на менаџирање на неструктурирани податоци, апликацијата подржува неколку функционалности:

- Генерирање на вештини од текст — системот овозможува автоматско генерирање и процесирање на текст кој содржи опис на вештини. Системот предложува листа од вештини со нивоа кои може да се додадат во системот. Вештините кои веќе постојат во системот се детектирани за да не се генерираат дупликати.
- Генерирање и детектирање на промени во вештини на вработени — менаџерите на системот може да прикачат резиме на некој вработен кое што ќе биде обработено и споредено со неговите постоечки вештини, од кои што

некои може да се подобрени, ослабнати или пак можеби сосема нови и непостоечки во системот.

- Генерирање на огласи за работа – менаџерите на системот може да изберат низа од вештини кои постојат во системот и да побараат од системот да генерира текст за оглас за работна позиција.
- Автоматско процесирање на прикачени резимеа на огласите – сите корисници кои аплицираат на огласите во системот со прикачено резиме, се процесираат веднаш, при што автоматски се генерира оценка за тоа колку корисникот е соодветен за конкретната позиција

Со имплементацијата на овие функционалности, апликацијата претставува комплетно решение за автоматизирано менаџирање на вештини и човечки ресурси, базирано на обработка на неструктурирани податоци. Таа овозможува значително намалување на мануелната работа, побрзо донесување одлуки и подобра искористеност на достапните информации.

3.2. Кориснички улоги и автентикација

Апликацијата овозможува пристап на 3 различни корисници: менаџери, вработени и нерегистрирани корисници. Разликата помеѓу менаџерите и вработените е контролирана со ролји кои што се поставени на корисниците. Секоја од овие улоги има различни дозволи и пристап до функционалности, со што се обезбедува соодветна контрола и безбедност на системот.

3.2.1. Нерегистрирани корисници

Овие корисници можат да пристапат до апликацијата без креирање на корисничка сметка. Главните функционалности кои се достапни за нив се:

- Прелистување на огласи за работа – корисниците може да ги разгледаат огласите за работа во системот
- Аплицирање на работна позиција – корисниците може да аплицираат на оглас за работа со тоа што ќе внесат основни податоци и ќе прикачат резиме

Нерегистрираните корисници не можат:

- Да пристапат до својот профил или до листата на корисници
- Да пристапат до листата на вештини и детали за секое ниво
- Да ја користат функционалноста за процесирање на текст/документ за

генерирање на вештини

3.2.2. Вработени

Овие корисници се регистрирани во системот и ја имаат рољата ‘EMPLOYEE’. Главните функционалности кои се достапни за нив се:

- Преглед на својот профил и поседуваните вештини
- Преглед на сите вработени во системот
- Преглед на сите вештини во системот
- Преглед на сите огласи за работа

Вработените корисници не можат:

- Да ја користат функционалноста за процесирање на текст/документ за генерирање на вештини
- Да ги прегледуваат аплицираните кандидати на огласите за работа

3.2.3. Менаџери

Корисниците кои што се менаџери имаат целосен пристап до сите функционалности на апликацијата по најавувањето. Тие можат:

- Да ги прелистаат сите вработени во системот, да го разгледаат нивниот профил и менаџираат со вештините
- Да процесираат резиме на вработен и ги искористат генерираните вештини за унапредување/додавање на вештини
- Да ги прелистаат сите вештини во системот и да ја користат функционалноста за генерирање на нови вештини
- Да генерираат нови огласи за работа и да ги прелистаат сите огласи во системот

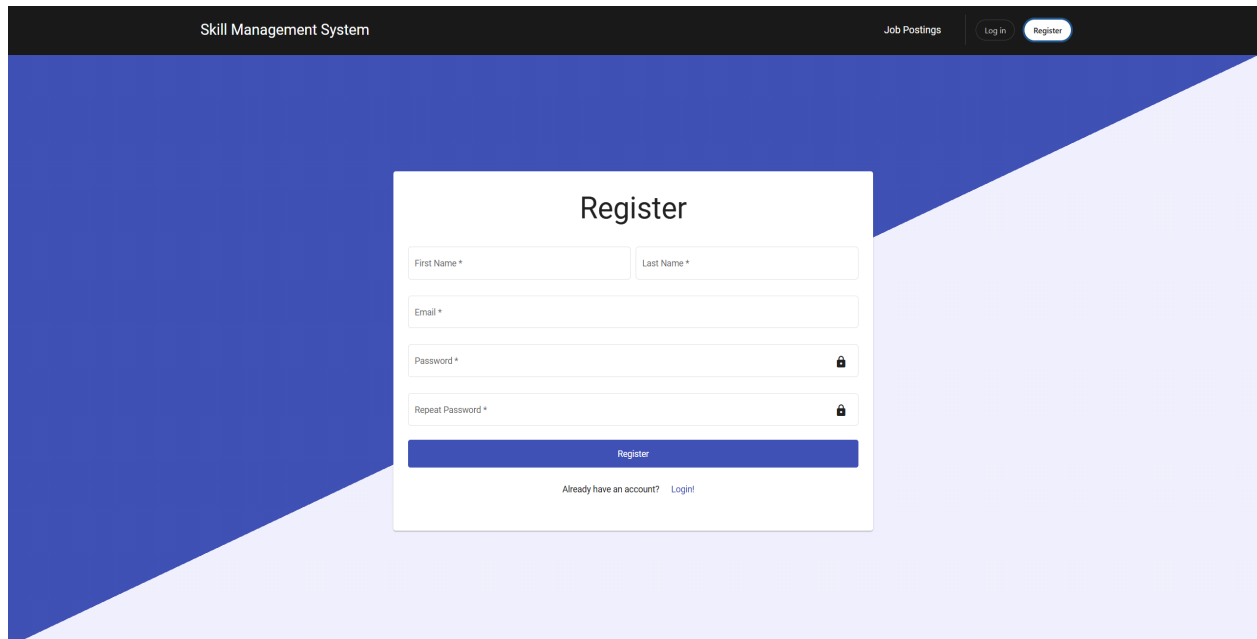
3.3. Функционалности

3.3.1. Регистрација и најавување на корисник

Во оваа секција се опишани функционалностите поврзани со автентикација на корисниците во апликацијата. Овие страни претставуваат клучен дел од системот, бидејќи овозможуваат безбеден пристап и идентификација на корисниците според нивните улоги. Преку овие механизми се обезбедува контролирано користење на функционалностите и заштита на податоците.

3.3.2. Регистрација на корисник

Регистрацијата овозможува креирање на нов кориснички профил во системот. При овој процес, корисникот треба да внесе основни податоци како име, презиме, e-mail адреса и лозинка. По успешно креирање на профилот, корисникот може да се најави и да започне со користење на апликацијата.

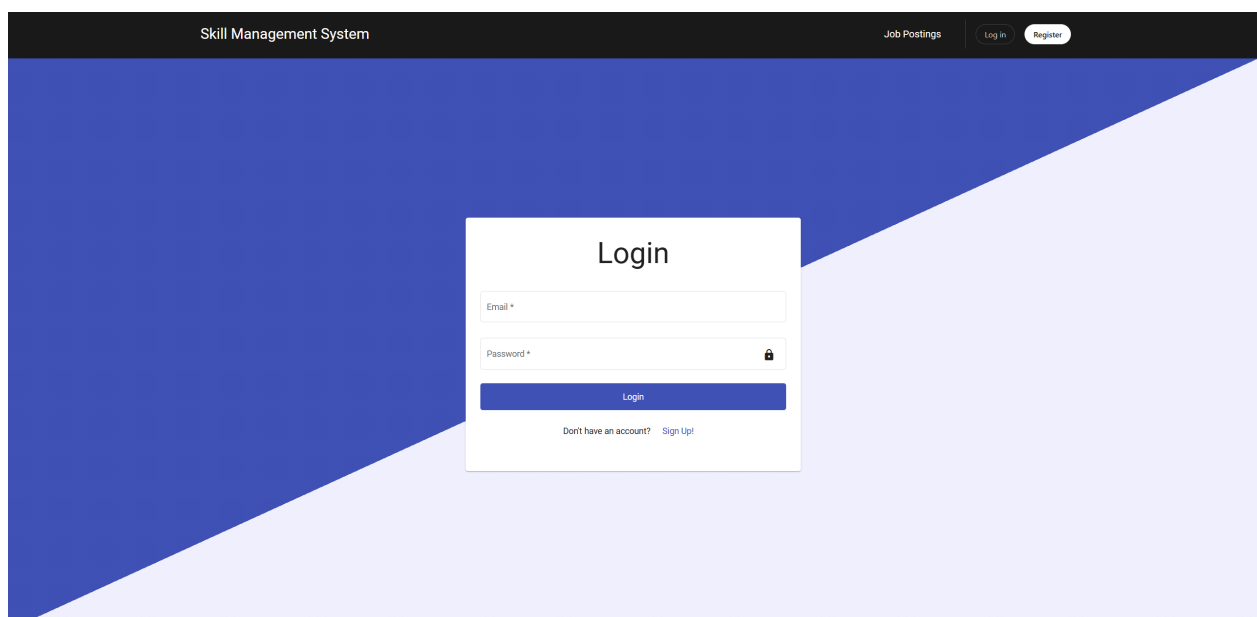


The screenshot shows the 'Register' page of the 'Skill Management System'. The page has a dark blue header with the system name and navigation links for 'Job Postings', 'Log in', and 'Register'. The main content area has a blue and light blue background. A white registration form is centered, titled 'Register'. It contains input fields for 'First Name *', 'Last Name *', 'Email *', 'Password *', and 'Repeat Password *'. The password fields have a lock icon. Below the fields is a blue 'Register' button. At the bottom of the form, there is a link 'Already have an account? Login!'.

Слика 3. Изглед на Register page

3.3.3. Најава на корисник

Процесот на логирање овозможува постоечките корисници да пристапат до системот преку внесување на корисничко име и лозинка. По успешна автентикација, корисникот се пренасочува кон страната за преглед на неговиот кориснички профил.

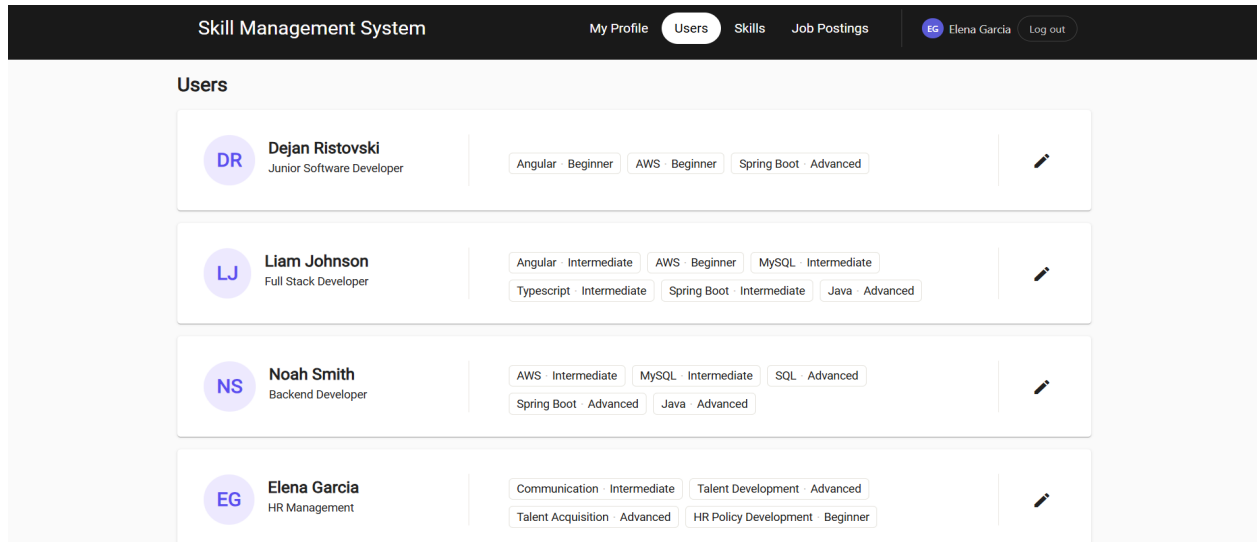


The screenshot shows the 'Login' page of the 'Skill Management System'. The page has a dark blue header with the system name and navigation links for 'Job Postings', 'Log in', and 'Register'. The main content area has a blue and light blue background. A white login form is centered, titled 'Login'. It contains input fields for 'Email *' and 'Password *'. The password field has a lock icon. Below the fields is a blue 'Login' button. At the bottom of the form, there is a link 'Don't have an account? Sign Up!'.

Слика 4. Изглед на Login page

3.3.4. Преглед на корисници во системот

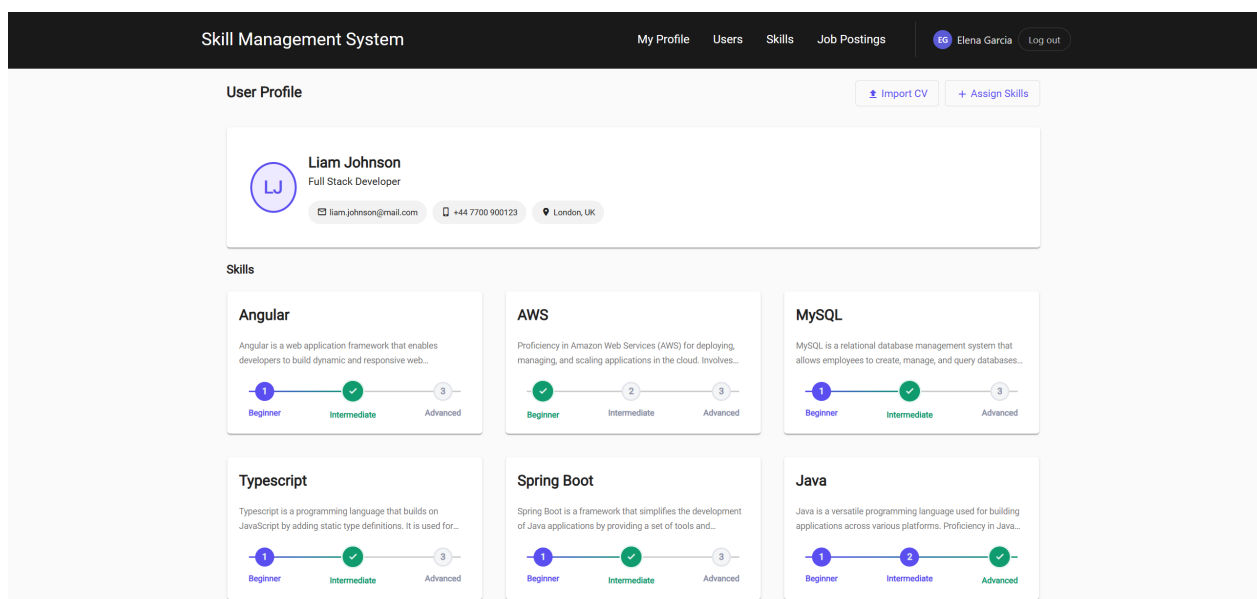
Страницата за преглед на корисници овозможува увид на сите корисници кои што се регистрирани во системот. Овде може да се видат основните информации на корисникот како и вештините кои што му се назначени. Доколку корисникот сака да види повеќе детали може да кликне на некој корисник и системот ќе го однесе на профилот на корисникот.



Слика 5. Изглед на страницата за преглед на корисниците во системот

3.3.5. Преглед на кориснички профил

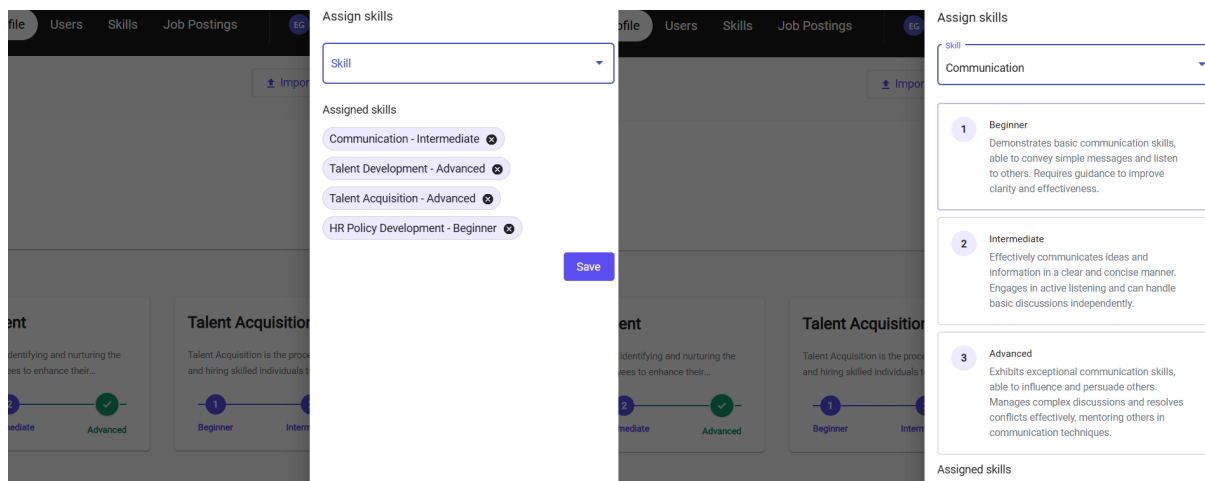
Апликацијата овозможува преглед на корисничкиот профил на секој вработен каде што се прикажани основните информации за корисникот како што се името, презимето, позицијата, е-маил, телефонски број и локација. Освен тоа прикажани се и вештините кои што ги поседува корисникот со тоа што е назначено нивото на познавање за секоја вештина.



Слика 6. Изглед на кориснички профил

3.3.6. Менаџирање на вештини

За секој профил може да се менаџираат вештините преку користење на копчето за assign skills со кое што се отвара drawer. Тука можеме да избришеме вештини за корисникот или пак да додадеме некои нови.



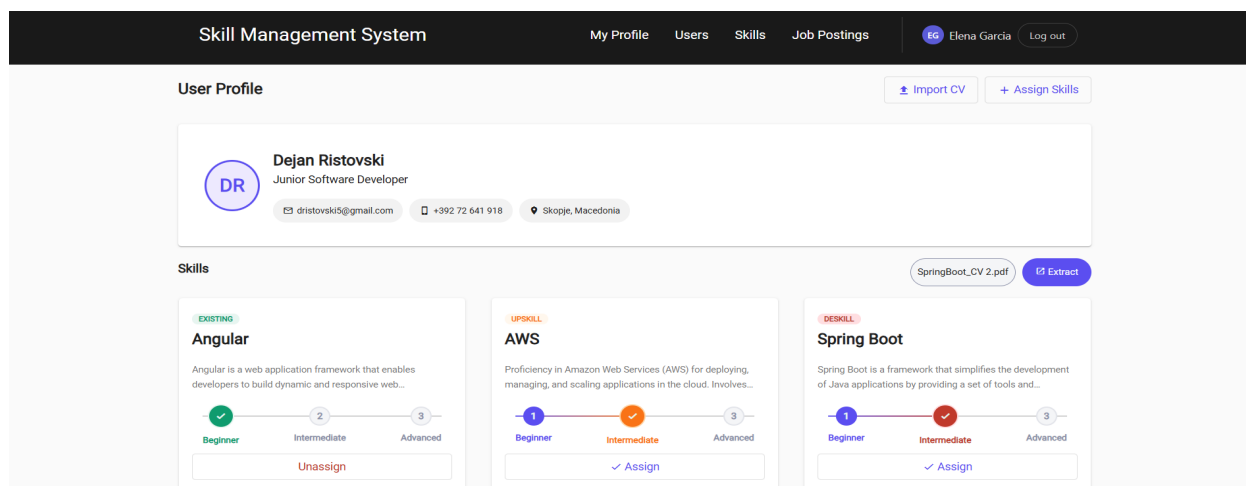
Слика 7. Изглед на drawer за менаџирање на вештини

3.3.7. Процесирање на резиме и менаџирање на унапредени вештини

На истата страна, менаџерите во системот може да прикачат резиме и да побараат системот да го процесира. Од процесираниот резиме, системот ќе изгенерира неколку вештини од кои може да се во неколку состојби:

- Existing – вештината веќе постои во системот
- Upskill – вештината веќе постои во системот и е подобрена
- Downskill – вештината веќе постои во системот и е ослабната
- New – нови вештини кои не постојат во системот

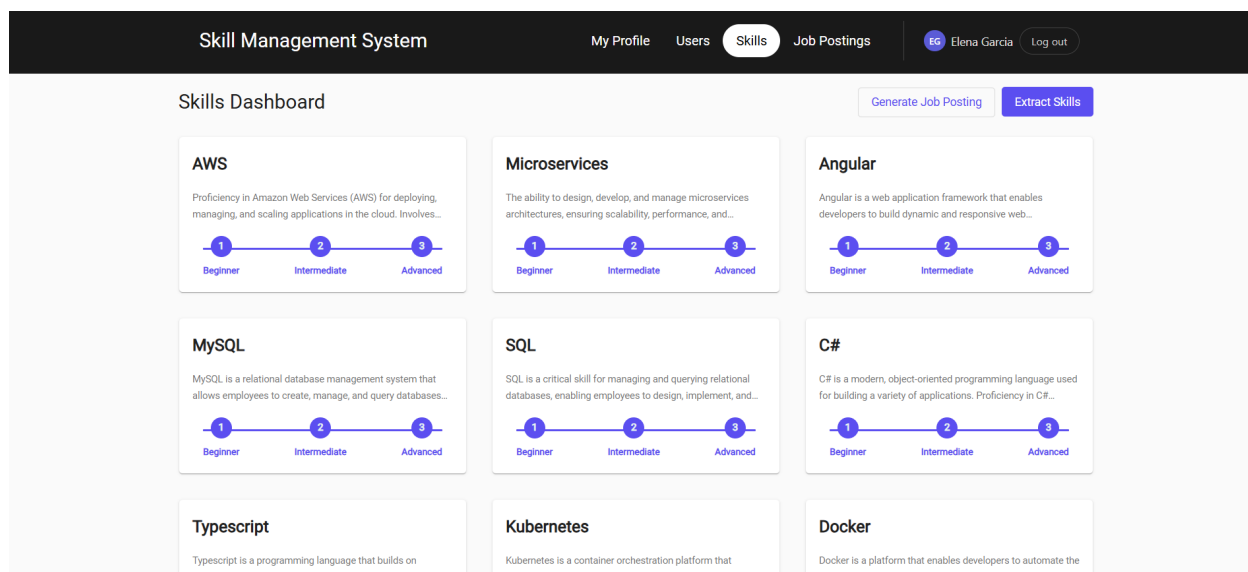
Корисникот може да одбере да ја сочува вештината и да ја додаде на корисникот. Секоја состојба на вештините е во посебна боја со што визуелно се покажува напредокот на вработениот.



Слика 8. Изглед на кориснички профил во состојба на процесирање на резиме

3.3.8. Преглед на вештини (Skills dashboard)

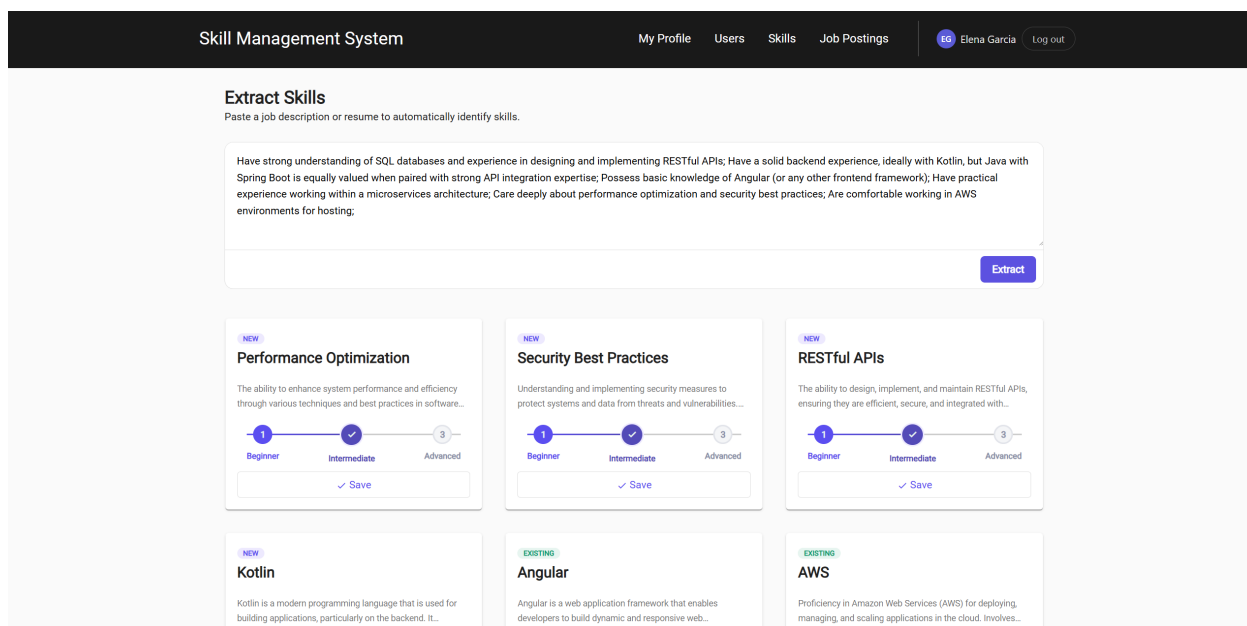
Skills Dashboard претставува централизирана страница за преглед и управување со вештините на корисниците. Секоја вештина е прикажана во посебна картичка со краток опис и визуелен индикатор за нивото на познавање (Beginner, Intermediate, Advanced). Преку јасно обележани бои и скала, корисникот лесно може да го следи својот напредок и да ги идентификува областите за подобрување. Дополнително, достапни се опции за генерирање огласи за работа и екстракција на вештини, што го прави системот практичен за управување со компетенции.



Слика 9. Изглед на главната страница за преглед на вештини

3.3.9. Процесирање на текст и менаџирање на вештини

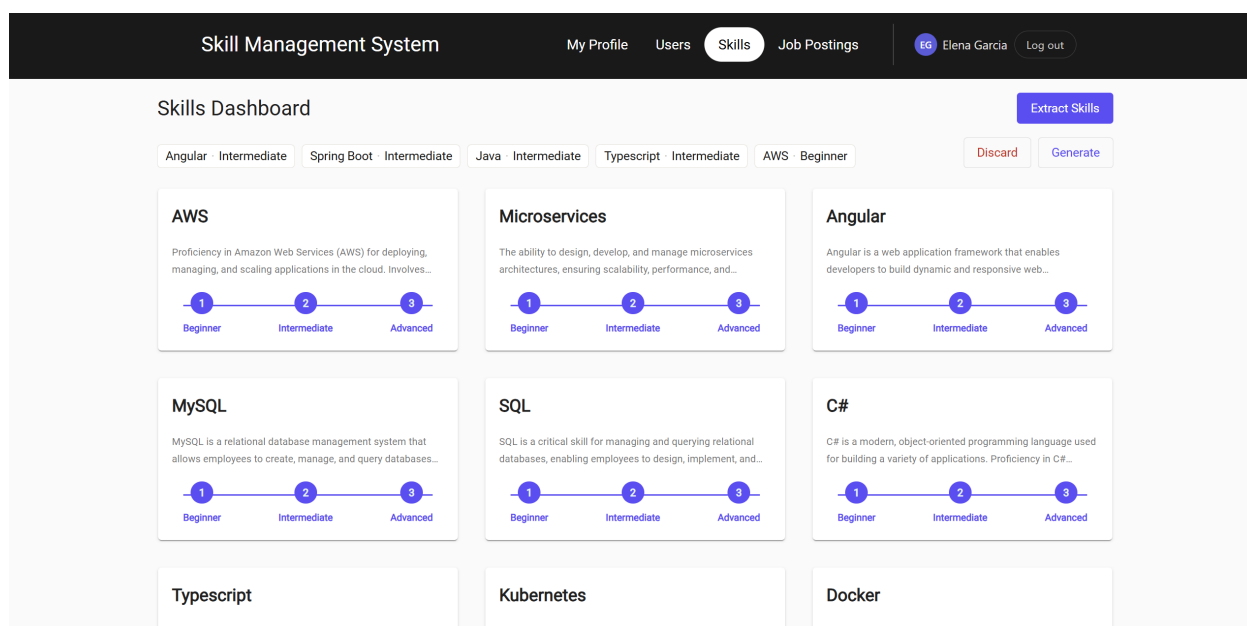
Страницата за екстракција на вештини овозможува автоматско препознавање на релевантни вештини преку внес на текст, како што се опис на работа или CV. Корисникот внесува текст во предвиденото поле, по што системот ги анализира податоците и генерира листа на препознаени вештини. Секоја вештина е прикажана во картичка со опис, ниво на познавање и можност за зачувување. На секоја вештина е означено нивото на познавање кое е детектирано од процесираниот текст. Со притискање на некое од нивоата на познавање, се отвара drawer каде што може да се видат повеќе информации за секоја вештина. Вештините кои ќе се генерираат, а веќе постојат во системот ќе се означат како 'Existing' и системот ќе ја спречи можноста да се додадат во системот. Ваквиот пристап значително го олеснува процесот на додавање и ажурирање на вештини, заштедува време и ја зголемува прецизноста при менаџирање на компетенциите.



Слика 10. Изглед на страница за менаџирање на вештини преку процесирање на текст

3.3.10. Генерирање на оглас за работа

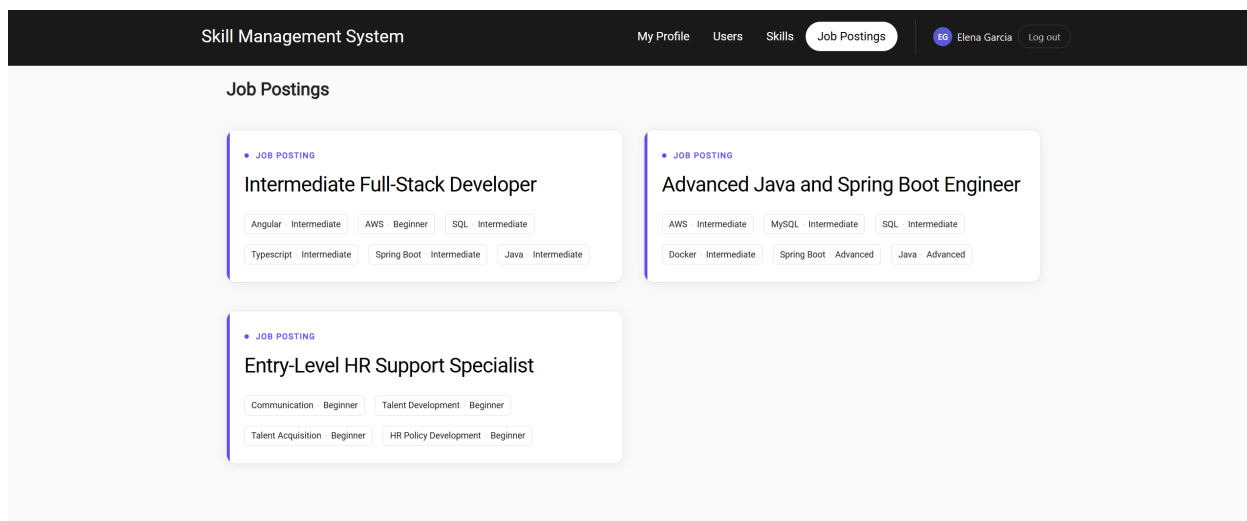
Функционалноста за генерирање оглас за работа овозможува испраќање на селектираните вештини заедно со нивните нивоа на познавање кон модел кој автоматски генерира текст за оглас. Со кликање на “Generate”, системот ги собира избраните вештини (пр. Angular – Intermediate, AWS – Beginner) и ги структурира како влезни податоци. Овие податоци потоа се проследуваат до моделот, кој врз основа на нив креира соодветен и конзистентен опис на работната позиција, вклучувајќи барања, одговорности и потребни квалификации.



Слика 11. Процес на генерирање на оглас

3.3.11. Преглед на огласи за работа

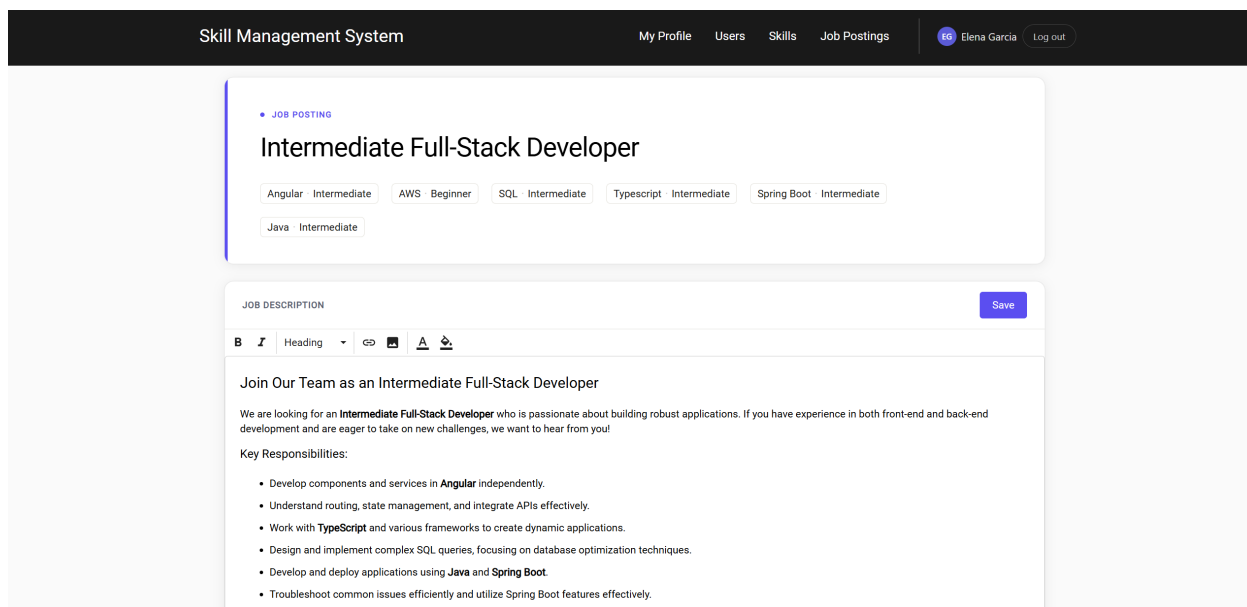
Страницата за преглед на огласи за работа претставува клучен дел од системот за управување со вештини, овозможувајќи им на корисниците лесен и интуитивен пристап до достапните работни позиции. Како што е прикажано на сликата, огласите се организирани во визуелно јасни картички, при што секој оглас содржи наслов на позицијата и листа на потребни вештини со соодветно ниво на познавање.



Слика 12. Изглед на страна за преглед на огласи за работа

3.3.12. Преглед и менаџирање на оглас за работа

Кај страницата за преглед и менаџирање на поединечен оглас за работа овозможува детален увид во сите информации поврзани со избраната позиција, вклучувајќи опис на обврските и потребните вештини. Дополнително, корисникот има можност за уредување и ажурирање на огласот преку користење на алатките за уредување на текст.



Слика 13. Изглед на страна за менаџирање на оглас за работа

3.3.13 Аплицирање на оглас за работа

Формата за аплицирање на оглас овозможува корисниците едноставно да ја поднесат својата апликација за избраниот оглас за работа преку пополнување на основни лични податоци и прикачување на своето CV. Прикаченото CV се праќа до системот каде што се процесира автоматски и се одредува оценка во зависност од вештините кои што се назначени на огласот.

• Develop and deploy applications using **Java** and **Spring Boot**.
• Troubleshoot common issues efficiently and utilize Spring Boot features effectively.

Required Skills:

- **Intermediate Level in Angular:** Can independently develop components and services.
- **Intermediate Level in SQL Databases:** Strong understanding with complex query implementation.
- **Intermediate Level in TypeScript:** Develop applications and integrate with JavaScript libraries.
- **Intermediate Level in Java and Spring Boot:** Adept in object-oriented programming, databases, and API integration.

Preferred Qualifications:

- Basic understanding of **AWS** services and functionalities.
- Ability to deploy simple applications using AWS with guidance.

If you are eager to improve your skills and contribute to exciting projects, apply today!

Apply Here!

First Name: Last Name:

Email:

SpringBoot_CV 1.pdf

Слика 14. Изглед форма за аплицирање на оглас

3.3.14 Преглед на корисници кои аплицирале на оглас

Оваа страница прикажува кандидати кои аплицирале на огласот, заедно со нивната процентуална усогласеност со бараните вештини. Процентот се пресметува врз основа на совпаѓањето на вештините од CV-то со барањата на позицијата. Повисок процент значи подобра релевантност на кандидатот. Во некои случаи, процентот може да биде и над 100%, што укажува на натпросечна квалификуваност.

Candidate	Score
DW Daniel Wilson daniel.wilson@mail.com	54%
LA Lara Anderson lara.anderson@mail.com	45%
MB Mateo Brown mateo.brown@mail.com	0%

Слика 15. Преглед на корисници кои аплицирале на оглас

4. Заклучок

Во овој труд беше претставено современо софтверско решение за автоматизирано процесирање на неструктурирани текстуални податоци, со фокус на екстракција и менаџирање на вештини на вработени. Преку интеграција на големи јазични модели, векторски бази на податоци и модерни backend и frontend технологии, беше демонстрирано како сложени и временски интензивни процеси можат да се автоматизираат и значително да се подобрат.

Предложениот процесен тек, базиран на повеќе специјализирани агенти, овозможува прецизно мапирање на текстуални податоци во структурирани информации. Овој пристап се покажа како поефикасен и пофлексибилен во споредба со традиционалните методи базирани исклучиво на keyword или embedding споредба. Со користење на RAG архитектура, системот успева да го надмине ограничувањето на статичкото знаење на моделите и да обезбеди контекстуално релевантни резултати.

Имплементираната апликација потврдува дека ваквиот пристап има практична вредност во реални сценарија, особено во доменот на човечки ресурси. Автоматското генерирање и ажурирање на вештини, како и евалуацијата на кандидати, овозможуваат побрзо и пообјективно донесување одлуки. Дополнително, системот придонесува за намалување на мануелната работа и подобрување на ефикасноста при управување со голем број на податоци.

Сепак, постојат и одредени ограничувања. Точноста на системот во голема мера зависи од квалитетот на дефинираните вештини и нивните нивоа, како и од правилната конфигурација на prompt-овите. Исто така, постои можност за грешки при интерпретација на комплексни или нејасни текстови. Овие предизвици отвораат простор за понатамошно истражување и подобрување каде што може да се проба:

- Користење на поголем модел – поголемите модели овозможуваат подобри перформанси со тоа што имаат поголем број на параметри и понов корпус на знаење.
- Користење на NER модел – NER моделите може да се искористат за да се намалат API повиците до OpenAI и со тоа да се намалат трошоците. Дополнително, доколку се реши да се користи свој хостиран LLM, тогаш ќе ги зголемат и перформансите.

- Креирање на генерички систем за процесирање на податоци – оваа апликација е конкретно базирана на доменот на менаџмент на човечки ресурси. Доколку се смени структурата на апликацијата да поддржува поставување на генерички правила, може да се постигне и систем кој врши автоматизација за други домени.

За крај, овој проект претставува значаен чекор кон модернизација на процесите за обработка на неструктурирани податоци и нивна примена во реални деловни сценарија. Преку комбинирање на најновите достигнувања во областа на вештачката интелигенција и софтверското инженерство, се демонстрира потенцијалот за изградба на интелигентни системи кои не само што автоматизираат, туку и унапредуваат комплексни процеси како што е менаџирањето на човечки ресурси. Ваквиот пристап отвора нови можности за понатамошна примена и развој, со цел креирање на поефикасни, скалабилни и интелигентни решенија во различни индустрии.

5. Библиографија

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin, "Attention Is All You Need". Available: <https://arxiv.org/abs/1706.03762> (пристапено декември 2025)
- [2] PostgreSQL, "PostgreSQL: The World's Most Advanced Open Source Relational Database," [Online]. Available: <https://www.postgresql.org/>.(пристапено март 2026)
- [3] Tomas Mikolov, Kai Chen, Greg Corrado, Jeffrey Dean, "Efficient Estimation of Word Representations in Vector Space". Available: <https://arxiv.org/abs/1301.3781> (пристапено март 2026)
- [4] Laurens van der Maaten, Geoffrey Hinton, "Visualizing Data using t-SNE". Available: <https://www.jmlr.org/papers/volume9/vandemaaten08a/vandemaaten08a.pdf>
- [5] Pivotal Software. (2023). *Spring Boot Reference Guide*. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>(пристапено март 2026)
- [6] Angular Team. (2026). *Angular Documentation*. Available: <https://angular.io/docs> (пристапено март 2026)